Interaction Portal (v2)

What is the Interaction Portal? (Overview for Human Users)

The **Interaction Portal** is an interactive, browser-based collaborative workspace where human participants work together with a team of artificial intelligence (AI) agents to solve realistic business and marketing strategy challenges. In this study, you step into the role of a team lead or strategic decision-maker addressing client briefs--such as launching an educational campaign, redesigning a brand, or planning an event.

How Users Use It: - **Meet the AI Team:** You collaborate with four specialized AI domain experts--**Nia** (Creative Concept & Copywriting), **Theo** (Audience & Market Research), **Rhys** (Channels & Content Distribution), and **Mira** (Budget, Feasibility & Metrics)--coordinated by an **AI Orchestrator** that synthesizes team progress. - **Interactive Live Chat:** You communicate with the team via a familiar group chat interface, directing questions to specific specialists using `@Nia`, `@Theo`, `@Rhys`, `@Mira`, or broadcasting questions to `@team`. - **Real-Time Guidance & Feedback Panel:** As you and the agents brainstorm, an intelligent side panel continuously summarizes team progress, diagnoses what requirements might be missing or unverified, and provides strategic coaching. - **Creating the Deliverable:** Depending on your experimental condition, you either observe the AI drafting the recommendation card (Passive), review and approve/reject/modify proposed card points using Keep, Cut, and Send-back controls (Evaluative), or write the strategic card yourself with AI-suggested inspirations (Generative). In the baseline condition (No-AI), you solve the challenges independently using a structured digital Reference Pack containing all source materials. - **Reflections & Breaks:** Seven clients are presented in total: six collaborative tasks with the team, and the final client alone. After submitting your strategic recommendation for each client, you complete short reflection check-ins. A structured break occurs after the third client to ensure you stay refreshed.

Why This Study Matters (The Research Objective): As generative AI transforms modern workplaces, teams increasingly involve humans working alongside multiple autonomous AI agents. This research portal investigates **how different levels of human control (agency)** and **different styles of AI management feedback** impact problem-solving quality, critical thinking, creative output, and human satisfaction. By tracking card edits (debounced diffs after 700 ms), messages, review decisions, attention telemetry (window blur/focus, scroll samples, accordion fold interactions), and time spent on reference materials in a secure, privacy-preserving event log, this platform provides scientific insights into how future AI systems can be designed to empower human workers rather than overwhelm or replace them.

Quick Start: How to Run the Application (Step-by-Step Guide)

This guide walks you step-by-step through setting up and running the Interaction Portal on your local machine. It is written especially for users and researchers who may have little or no programming experience.

System Requirements & Prerequisites

Before you begin, ensure you have the following installed on your computer:

1. **Git** (Code Download & Version Control): - Used to download the code repository to your computer. - If you do not have Git, download and install it for free from git-scm.com (using the default installer options).
2. **Anaconda or Miniconda** (Recommended Python Environment Manager): - Manages Python and project dependencies cleanly in an isolated sandbox, avoiding any conflicts with existing software. - If you do not have it installed, download Miniconda (lightweight, recommended) or Anaconda. - *(Alternative)*: If you already have standard Python (version 3.10, 3.11, or 3.12) installed from python.org, that also works with standard `venv`.
3. **A Modern Desktop Web Browser**: - Google Chrome, Microsoft Edge, Mozilla Firefox, or Apple Safari. - *Note:* The experiment requires a standard desktop or laptop screen (minimum viewport width 900px). Mobile devices and tablets are blocked by default to preserve experimental control.
4. **(Optional) AI Provider API Key (Claude, ChatGPT / OpenAI, DeepSeek, GLM, or Qwen)**: - **Only required for Real AI Mode**. The portal supports API keys from **Claude** (Anthropic), **ChatGPT** (OpenAI), **DeepSeek**, **GLM** (Zhipu AI), and **Qwen** (Alibaba Cloud DashScope). If you want to explore the application, test conditions, or conduct local demonstrations, **no API key is needed**--the portal includes a free, fully deterministic offline mock AI out of the box!

Step 1: Clone or Download the Repository

Open your terminal or command prompt and clone the repository:

```
git clone https://github.com/your-username/multi_agent_chat.git
```

> **Note**
> **Downloaded as a ZIP file?** If you downloaded a `.zip` file instead of using Git, extract the archive into a folder on your computer (for example, `D:\Desktop\Git\multi_agent_chat` or `C:\Users\YourName\multi_agent_chat`).

Step 2: Open Anaconda Prompt (or Terminal)

- **On Windows (using Anaconda / Miniconda - Recommended):** 1. Click the Windows Start menu or press the Windows key. 2. Type **"Anaconda Prompt"** (or **"Miniconda Prompt"**). 3. Click to launch the prompt window.
- **On Windows (using PowerShell or Command Prompt):**
- Press the Windows key, search for **"PowerShell"** or **"cmd"**, and press Enter.
- **On macOS / Linux:**
- Open the **Terminal** application from Applications or Spotlight.

Step 3: Navigate to the `interaction_portal` Folder

Use the `cd` (change directory) command to navigate into the `interaction_portal` subfolder where the project files reside.

On Windows:

```
# If your folder is on a different drive (e.g., D:), switch drives first:
d:

# Then change directory to the interaction_portal folder:
cd D:\Desktop\Git\multi_agent_chat\interaction_portal
```

On macOS / Linux:

```
cd ~/path/to/multi_agent_chat/interaction_portal
```

> **Tip**
> **Verify your directory:** Type `dir` (on Windows) or `ls` (on macOS/Linux) and press Enter. You should see folders such as `app`, `config`, `logs`, and files such as `requirements.txt`. If you see them, you are in the correct directory.

Step 4: Create and Activate a Dedicated Virtual Environment

A virtual environment keeps the project's dependencies completely isolated from the rest of your system.

Option A: Using Conda (Recommended if you installed Anaconda / Miniconda)

```
# 1. Create a fresh environment named "portal" with Python 3.11:
conda create -n portal python=3.11 -y

# 2. Activate the environment:
conda activate portal
```

Your prompt line will now show `(portal)` at the beginning, confirming that the environment is active.

Option B: Using Standard Python `venv` (If not using Conda)

On Windows (Command Prompt / Anaconda Prompt):

```
python -m venv .venv
.venv\Scripts\activate
```

On Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

(If PowerShell displays an `ExecutionPolicy` error, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` and then run the activate command again).

On macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Step 5: Install Project Dependencies

Make sure your environment is activated (you should see `(portal)` or `(.venv)` in your terminal), then install the required Python libraries:

```
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, Jinja2, Pydantic, PyYAML, OpenAI client, and terminal color support. The process usually takes less than a minute.

Step 6: Launch the Application

You can launch the Interaction Portal in either **Standalone Test Mode** (zero setup, no API key) or **Real AI Mode** (using live models from Claude, ChatGPT / OpenAI, DeepSeek, GLM, or Qwen).

Mode A: Standalone Test Mode (Recommended for Beginners & First-Time Testing)

100% Free * Works completely offline * No API key needed * Interactive condition launcher

In this mode, the portal uses built-in, deterministic simulated AI responses. It launches an interactive browser **Test Launcher** where you can select any of the 10 experimental conditions (`PAS-COORD`, `EVA-TASK`, `GEN-DEV`, `NOAI`, etc.) with a single click--no Qualtrics account or API keys required.

1. **Start the server:**

- **In Anaconda Prompt or Windows Command Prompt (`cmd.exe`):** `cmd set PORTAL_STANDALONE=1 python -m app.main`
- **In Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" python -m app.main`
- **On macOS / Linux:** `bash export PORTAL_STANDALONE="1" python -m app.main`

2. **Open the portal in your browser:** Open Google Chrome, Microsoft Edge, or Firefox and navigate to: `text http://localhost:8000/` *(or `http://127.0.0.1:8000/`)*

3. **Experience the workflow:** - The **Interaction Portal Test Launcher** will appear on your screen. - Choose any experimental condition from the dropdown or click one of the quick-launch buttons. - An interactive participant session starts immediately! You can chat with specialists (`@Nia`, `@Theo`, `@Rhys`, `@Mira`, `@team`), inspect the live Orchestrator coaching panel, review or author deliverable cards, and complete the post-task reflection check-ins. - At the end of the session, the completion code is displayed directly on the exit screen without redirecting to Qualtrics.

Mode B: Real AI Mode (Live LLM with API Key: Claude, ChatGPT, DeepSeek, GLM, Qwen)

Multi-Provider Support * Real-Time Chat Streaming * Pinned Reproducible Models

In this mode, the portal connects to live frontier AI models from your chosen provider. Specialists brainstorm dynamically, answer tailored questions, and stream tokens live.

Supported Providers & Configuration Reference

Provider	`PORTAL_LLM_PROVIDER`	Primary API Key Env	Fallback Env Vars	Default Model	Default Base URL
Claude (Anthropic)	`claude`	`ANTHROPIC_API_KEY`	`CLAUDE_API_KEY`, `PORTAL_LLM_API_KEY`	`claude-haiku-5-5`	`https://api.anthropic.com`
ChatGPT (OpenAI)	`openai` *(or `chatgpt`)*	`OPENAI_API_KEY`	`CHATGPT_API_KEY`, `PORTAL_LLM_API_KEY`	`gpt-5.4-mini`	`https://api.openai.com/v1`
DeepSeek	`deepseek`	`DEEPSEEK_API_KEY`	`PORTAL_LLM_API_KE`	`deepseek-flash`	`https://api.deepseek.com`

Provider	PORTAL_LLM_PROVIDER	Primary API Key Env	Fallback Env Vars	Default Model	Default Base URL
GLM (Z.ai, formerly Zhipu AI)	`glm` *(or `zhipu`, `zai`)*	`ZAI_API_KEY`	`GLM_API_KEY`, `ZHIPUAI_API_KEY`, `BIGMODEL_API_KEY`, `PORTAL_LLM_API_KEY`	`glm-5.3-flash`	`https://api.z.ai/api/paas/`
Qwen (Alibaba Cloud Model Studio, also called DashScope)	`qwen` *(or `dashscope`)*	`DASHSCOPE_API_KEY`	`QWEN_API_KEY`, `PORTAL_LLM_API_KEY`	`qwen3.8-flash`	`https://dashscope-intl.ali`

1. Obtain an API Key from Your Preferred Provider

- **Claude (Anthropic):** Sign up at console.anthropic.com and create an API key under **API Keys**.
- **ChatGPT (OpenAI):** Sign up at platform.openai.com and generate a key under **API Keys** (starts with `sk-...`).
- **DeepSeek:** Sign up at platform.deepseek.com and generate an API key under **API Keys**.
- **GLM (Z.ai, formerly Zhipu AI):** Sign up at z.ai (or bigmodel.cn) and create an API key in the developer console.
- **Qwen (Alibaba Cloud Model Studio / DashScope):** Sign up at dashscope.aliyun.com (or the International portal) and generate a DashScope API key.

2. Launch the Application with Your Provider & Key

Choose your provider and run the corresponding commands below:

Option 1: Claude (Anthropic)

- **Windows Command Prompt (`cmd.exe`) / Anaconda Prompt:** `cmd set PORTAL_STANDALONE=1 set PORTAL_LLM_PROVIDER=claude set ANTHROPIC_API_KEY=your-anthropic-key-here python -m app.main`
- **Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" env:PORTAL_LLM_PROVIDER="claude" env:ANTHROPIC_API_KEY="your-anthropic-key-here" python -m app.main`
- **macOS / Linux:** `bash export PORTAL_STANDALONE="1" export PORTAL_LLM_PROVIDER="claude" export ANTHROPIC_API_KEY="your-anthropic-key-here" python -m app.main`

Option 2: ChatGPT / OpenAI

- **Windows Command Prompt (`cmd.exe`) / Anaconda Prompt:** `cmd set PORTAL_STANDALONE=1 set PORTAL_LLM_PROVIDER=openai set OPENAI_API_KEY=sk-your-openai-key-here python -m app.main`
- **Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" env:PORTAL_LLM_PROVIDER="openai" env:OPENAI_API_KEY="sk-your-openai-key-here" python -m app.main`
- **macOS / Linux:** `bash export PORTAL_STANDALONE="1" export PORTAL_LLM_PROVIDER="openai" export OPENAI_API_KEY="sk-your-openai-key-here" python -m app.main`

Option 3: DeepSeek

- **Windows Command Prompt (`cmd.exe`) / Anaconda Prompt:** `cmd set PORTAL_STANDALONE=1 set PORTAL_LLM_PROVIDER=deepseek set DEEPSEEK_API_KEY=your-deepseek-key-here python -m app.main`
- **Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" env:PORTAL_LLM_PROVIDER="deepseek" env:DEEPSEEK_API_KEY="your-deepseek-key-here" python -m app.main`
- **macOS / Linux:** `bash export PORTAL_STANDALONE="1" export PORTAL_LLM_PROVIDER="deepseek" export DEEPSEEK_API_KEY="your-deepseek-key-here" python -m app.main`

Option 4: GLM (Z.ai, formerly Zhipu AI)

- **Windows Command Prompt (`cmd.exe`) / Anaconda Prompt:** `cmd set PORTAL_STANDALONE=1 set PORTAL_LLM_PROVIDER=glm set ZAI_API_KEY=your-zai-key-here python -m app.main` *(Note: `GLM_API_KEY` and `ZHIPUAI_API_KEY` are also automatically recognized as fallbacks).*
- **Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" env:PORTAL_LLM_PROVIDER="glm" env:ZAI_API_KEY="your-zai-key-here" python -m app.main`
- **macOS / Linux:** `bash export PORTAL_STANDALONE="1" export PORTAL_LLM_PROVIDER="glm" export ZAI_API_KEY="your-zai-key-here" python -m app.main`

Option 5: Qwen (Alibaba Cloud Model Studio / DashScope)

- **Windows Command Prompt (`cmd.exe`) / Anaconda Prompt:** `cmd set PORTAL_STANDALONE=1 set PORTAL_LLM_PROVIDER=qwen set DASHSCOPE_API_KEY=your-dashscope-key-here python -m app.main`
- **Windows PowerShell:** `powershell env:PORTAL_STANDALONE="1" env:PORTAL_LLM_PROVIDER="qwen" env:DASHSCOPE_API_KEY="your-dashscope-key-here" python -m app.main`
- **macOS / Linux:** `bash export PORTAL_STANDALONE="1" export PORTAL_LLM_PROVIDER="qwen" export DASHSCOPE_API_KEY="your-dashscope-key-here" python -m app.main` *(Note for China region users: to use the domestic endpoint instead of the international endpoint, set `DASHSCOPE_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1`).*

3. Optional Custom Model & Endpoint Overrides

You can override the model version or base URL for any provider without modifying configuration files: - **Override Model:** Set `PORTAL_LLM_MODEL` (e.g. `export PORTAL_LLM_MODEL="claude-3-5-sonnet-20241022"` or `export PORTAL_LLM_MODEL="gpt-4o"`). - **Override Endpoint / Base URL:** Set `PORTAL_LLM_BASE_URL` (e.g. for custom proxies, gateways, or local compatible servers). - **Universal API Key:** Alternatively, set `PORTAL_LLM_API_KEY`, which acts as a master key override across any active provider.

4. Access the Portal

- Open `http://localhost:8000/` in your browser.
- Select any condition in the launcher to begin interacting with live AI specialists powered by your chosen model provider.

> **Tip**
>
> **Conducting a Formal Study with Qualtrics?** When running a formal lab experiment where participants arrive from Qualtrics (with `?pid=...&condition=...`), set `PORTAL_STANDALONE=0` and configure production secrets. See [Section 4: Production Deployment & Database Administration](#4-production-deployment--database-administration) for full instructions on Qualtrics parameters, HMAC link signing, reverse proxies, and server-side completion verification.

Step 7: How to Stop the Application

When you are finished using or testing the portal: 1. Return to your terminal or Anaconda Prompt window. 2. Press **`Ctrl + C`** on your keyboard. 3. The server will shut down gracefully.

Beginner FAQs & Troubleshooting

- **"Unable to connect" or "This site can't be reached":**
- Ensure the terminal window where you ran `python -m app.main` is still open and displaying `Uvicorn running on http://127.0.0.1:8000`. Do not close that terminal while using the portal.
- **"Address already in use" (Port 8000 is occupied):**
- Another application or a previous portal session is using port 8000. You can easily run on port 8080 instead:
  - *Windows CMD / Anaconda Prompt:* `set PORTAL_PORT=8080` then `python -m app.main`
  - *Windows PowerShell:* `env:PORTAL_PORT="8080"` then `python -m app.main`
  - *macOS / Linux:* `export PORTAL_PORT="8080"` then `python -m app.main`
  - Then navigate to `http://localhost:8080/` in your browser.
- **"Desktop Viewport Required" warning:**
- The study design requires a desktop screen resolution (minimum 900 pixels width). If your browser window is tiled or resized to half-screen, maximize the window.
- **Where are the logs and experimental data saved?**
- All participant actions, chat messages, card edits, and timing metrics are automatically stored in an append-only SQLite database at `logs/portal.sqlite3`. You can view or export this data anytime (see Section 3.3: Logs & Telemetry).

---

1. Research Design & Conditions

The experiment employs a 3 x 3 + 1 between-subjects factorial design (10 conditions total): - **Participant Role** (3 levels): - **Passive (`PAS`)**: Read-only deliverable card; participant observes team drafting and submits. - **Evaluative (`EVA`)**: Participant acts as reviewer

using Keep, Cut, and Send-back controls on AI proposals. - **Generative (`GEN`)**: Participant writes card content directly, supported by AI suggestions (must exceed a minimum typed share, default 25%). - **Orchestrator Panel Feedback** (3 levels): - **Coordination (`COORD`)**: Status reports on specialist contributions and task assignments. - **Task-Focused (`TASK`)**: Status reports plus diagnosis of what is missing or unverified on the deliverable card. - **Developmental (`DEV`)**: Status, card diagnosis, plus recurring conceptual principles explaining why errors arise. - **No-AI Baseline (`NOAI`)**: - No chat, no agents, no panel. The participant works through the client tasks using a four-part collapsible **Reference Pack** containing the exact source materials held across specialists.

Condition Matrix

Code	Role	Panel Level	Team Chat	Card Editor Mode
`PAS-COORD`	Passive	Coordination	Active	Read-only
`PAS-TASK`	Passive	Task-Focused	Active	Read-only
`PAS-DEV`	Passive	Developmental	Active	Read-only
`EVA-COORD`	Evaluative	Coordination	Active	Keep / Cut / Send-back
`EVA-TASK`	Evaluative	Task-Focused	Active	Keep / Cut / Send-back
`EVA-DEV`	Evaluative	Developmental	Active	Keep / Cut / Send-back
`GEN-COORD`	Generative	Coordination	Active	Typing + AI suggestions (min 25% typed)
`GEN-TASK`	Generative	Task-Focused	Active	Typing + AI suggestions (min 25% typed)
`GEN-DEV`	Generative	Developmental	Active	Typing + AI suggestions (min 25% typed)
`NOAI`	None	None	Disabled	Reference pack + manual authoring

---

2. Architecture & Folder Structure

```
interaction_portal/
|--  app/                  # Application runtime and backend mechanics
|    |--  agents/          # LLM adapters, schemas, specialists, orchestrator, panel validator & PII safeguards
|    |--  harnesses/       # Automated verification harnesses (Harness A & Harness B)
|    |--  simulation/      # State coordination, card provenance, chat routing, practice
|    |--  static/          # Client-side CSS styling and vanilla JS (telemetry, editor, chat)
|    |--  templates/       # Jinja2 HTML templates for each experimental screen
|    |--  tools/           # Standalone maintenance CLIs (backup, verify_db, purge_pii)
|    |--  config_loader.py # Configuration parser, SHA-256 fingerprinting and validator
|    |--  event_store.py   # Append-only SQLite telemetry database engine with optimistic locking
|    |--  main.py          # FastAPI web application factory & routes
|    |--  state_machine.py # Forward-only page flow engine, Latin square balancer & authoritative timers
|    \--  terminal.py      # Cross-platform console color handling and ANSI filter stream
|--  config/               # Experiment content, prompts, condition matrix, UI text, and parameters
|    |--  conditions.yaml  # 10 condition definitions (role x panel level)
|    |--  config.yaml      # Master configuration file (UI text, timers, card fields, practice)
|    |--  content/clients/ # Client folders (C1..C7) with public briefs and private materials
|    \--  prompts/         # System prompts, orchestrator planning prompts, panel level templates
|         |--  panel_levels/ # Panel level templates (coordination.md, task_focused.md, developmental.md, _common.md
|         \--  panel_compress.md # Fallback compression prompt for panel length budget
|--  logs/                 # Telemetry database storage (portal.sqlite3)
|--  tests/                # Automated pytest unit and integration test suite (83 tests)
|--  md_to_pdf.py          # Documentation export utility (converts README.md to PDF)
|--  requirements.txt      # Python dependencies
\--  README.md             # This documentation file
```

---

3. Detailed Guide to Each Folder

3.1 `app/` -- Application Engine (For the Application Hoster)

Intention & Purpose: The `app/` directory contains the core FastAPI web application and simulation runtime. It manages client sessions, coordinates team chat streaming via NDJSON, enforces authoritative wall-clock server timers, tracks span-by-span deliverable card provenance, and logs fine-grained telemetry to SQLite.

Why Minimal Tweaking is Needed: The code in `app/` is designed as a generic, headless experimental motor. All experiment-specific wording, prompts, client briefs, agent personalities, and timing parameters are completely decoupled into `config/`. As an application hoster or sysadmin, you do not need to modify Python code in `app/` under normal operation. You only need to configure standard deployment environment variables (host, port, API keys, secret cookies) and run the service.

Core Submodules: - `app/agents/`: LLM client interfaces (`OpenAICompatClient` for ChatGPT, DeepSeek, GLM, Qwen; `ClaudeClient` for Anthropic Claude; and deterministic `MockClient`), Pydantic output schemas, keyword domain classifiers (`topics.py`), deterministic hidden-item disclosure detector (`disclosure.py`), orchestrator panel validator and proportional trim engine (`panel_validator.py`), and PII redaction safeguard (`pii.py`). - `app/simulation/`: Multi-client session manager (`session_manager.py`), card span provenance and role authoring rules (`card_provenance.py`), `@mention` chat routing and folded internal block grouper (`chat_router.py`), and scripted onboarding practice (`practice.py`). - `app/event_store.py`: Append-only SQLite data store equipped with SQL triggers that abort any `UPDATE` or `DELETE` statements on events, with optimistic version locking. - `app/state_machine.py`: Deterministic forward-only state progression engine ensuring participants cannot jump backwards or reset timers via browser reloads, backed by counterbalanced Latin square client assignment. - `app/main.py`: FastAPI application factory, signed session cookie management, streaming endpoints, and header-authenticated data export handlers. - `app/terminal.py`: Terminal execution initialization (`app.terminal.init_terminal()`), Windows Virtual Terminal (VT) color detection, and fallback ANSI filter stream. - `app/tools/`: Operational and administrative CLI tools: - `backup.py`: Online non-blocking SQLite database backup using SQLite's backup API, integrity check verification (`PRAGMA integrity_check`), gzip compression, and SHA-256 checksum manifest. - `verify_db.py`: Database verification script checking append-only SQL triggers, foreign keys, table schemas, row counts, and monotonic event ordering per session. - `purge_pii.py`: Participant text redaction and PII purging maintenance script that temporarily drops and re-establishes append-only SQL triggers inside an atomic logged transaction. - `app/static/` & `app/templates/`: Dependency-free vanilla JavaScript (attention telemetry, card editor, auto-expanding chat) and clean semantic Jinja2 HTML templates.

3.2 `config/` -- Study Configuration & Customization (For the Researcher / Coder)

Intention & Purpose: The `config/` directory is the single source of truth for the entire study. Everything participant-facing or model-facing lives here. By modifying these YAML and Markdown files, researchers can update screen text, rewrite prompts, adjust timers, or add entirely new client tasks without touching the Python backend.

How to Update and Tweak Configurations:

1. **Screen Text and Wording (`config/config.yaml` \to `pages:`):** - All text displayed to participants across all 11 screens is defined under the `pages` dictionary in `config/config.yaml`. - **Common captions:** `pages.common.continue`, `pages.common.submit`, `pages.common.unlocks_in`. - **Screen-specific instructions:**
    - `pages.orientation`: Orientation notes, privacy policy text, and requirement lists.
    - `pages.team`: Descriptions of specialists and the orchestrator card.
    - `pages.role`: Explanations for Passive, Evaluative, and Generative roles.
    - `pages.task`: Chat placeholders, unknown mention error templates, and card status notices.
    - `pages.checkin`: Post-task survey questions (`ci1`..`ci5`), Likert scale ranges, and multiple-choice options.
    - `pages.break`: Break instructions and early-finish button text.
    - `pages.exit`: Completion thank-you message and Qualtrics return instructions.
2. **Agent Behavior & System Prompts (`config/prompts/`):** - **Specialists (`config/prompts/specialists_system.md`):** System prompt defining the specialist's role, behavioral boundaries, JSON response schema, refusal rules for out-of-domain questions, and prohibitions on unprompted disclosure. - **Orchestrator Planning (`config/prompts/orchestrator_plan.md`):** Instructions guiding how the Orchestrator assigns tasks to specialists during opening rounds and coordination bursts. - **Orchestrator Panels (`config/prompts/panel_levels/`):**
    - `coordination.md`: Prompts the model to report specialist activity and contributions.
    - `task_focused.md`: Instructs the model to diagnose what is missing or unverified on the deliverable card against brief requirements.
    - `developmental.md`: Instructs the model to explain the recurring conceptual rationale for common errors.
    - `_common.md`: Shared panel constraints, including the strict 60-80 word budget, forbidden imperative language, and required section headers.
    - **Fallback Panel Compression (`config/prompts/panel_compress.md`):** Fallback compression prompt triggered if initial panel output exceeds the word budget.

3. **Client Tasks & Hidden Materials (`config/content/clients/`):** - Seven client tasks (`C1` through `C7`) are provided by default. Each folder contains:
    - `brief.md`: Public client brief with frontmatter `requirements: [...]` that participants read.
    - `private/nia.md`, `private/theo.md`, `private/rhys.md`, `private/mira.md`: Private materials held exclusively by each specialist.
    - **Hidden Items:** Marked in private files using HTML comments: `markdown <!-- hidden:C1-H1 --> The client privately stated that they will not accept any video shorter than 30 seconds. <!-- /hidden:C1-H1 -->` These markers allow the system to automatically track whether a specialist has disclosed a hidden constraint.
    - **Adding a New Client:** Create `config/content/clients/C8/` with `brief.md` and the 4 `private/*.md` files, then add `C8` to `clients.randomised` in `config/config.yaml`.
4. **Timers & Pacing (`config/config.yaml` \to `timers:`):** - `timers.task_seconds`: Duration of each client task (default `600` = 10 minutes). - `timers.break_seconds`: Mid-session break duration after client 3 (default `120` = 2 minutes). - `timers.orientation_min_seconds`: Minimum reading lockout before "Continue" unlocks (default `15`). - `timers.resume_gap_seconds`: Participant inactivity cutoff (default `900` = 15 minutes). After this window, re-entry is blocked as `abandoned_timeout`. - `testing.timer_scale`: Global speed multiplier for testing (e.g. set to `10.0` or `100.0` to shrink timers for rapid walkthroughs).
5. **Conditions Matrix (`config/conditions.yaml`):** - Maps condition codes (`PAS-COORD`, `EVA-TASK`, `GEN-DEV`, `NOAI`, etc.) to their active parameters (`role`, `panel_level`, `ai`).
6. **Card Editor & Anti-Gaming Safeguards (`config/config.yaml` \to `editor:`):** - `editor.allow_paste`: Toggles whether pasting into deliverable card fields is permitted (default `true`). When active, pasted text is logged (`card_paste` / `paste_chars`) and audited under strict typed share metrics (`typed_share_strict`), filtering out unoriginal filler and AI-copied content. - `editor.max_span_chars`: Maximum character cap per card field (default `600`), enforced with live UI counters and server-side validation.
7. **LLM Provider Compatibility Flags (`config/config.yaml` \to `llm:`):** - `llm.supports_stream_usage`: Configures whether the upstream endpoint accepts `stream_options` for token consumption. - `llm.token_param`: Selects either `max_tokens` or `max_completion_tokens` based on provider requirements. - `llm.send_temperature`: Controls whether temperature is sent in request payloads for newer model families with fixed temperatures.

3.3 `logs/` -- Telemetry & Database Storage (How to Open and Read Logs)

Intention & Purpose: The `logs/` folder stores the append-only SQLite database (`logs/portal.sqlite3`). This database logs every participant action, agent reply, card edit diff, scroll sample, window blur/focus event, and card submission with millisecond UTC timestamps.

Database Tables Overview:

- `events`: Core telemetry stream (`pid`, `timestamp`, `received_at`, `client_id`, `page_id`, `event_type`, `autopilot`, `metadata` JSON). SQL triggers prevent any modification or deletion.
- `sessions`: Participant session metadata (`pid`, `condition_code`, `started_at`, `ended_at`, `end_state`, `resume_count`, `completion_code`, `autopilot`, `version`, `resume_token`, `verify_count`, `config_sha256`, `state` JSON).
- `transcripts`: Full chronological chat transcripts per participant and client task, including private folded blocks.
- `panels`: Snapshots of every Orchestrator side-panel generated (`level`, `text`, `word_count`, `sections_present`, `validation_errors`, `retried`, `truncated`).
- `cards`: Final and intermediate deliverable cards with span-by-span author attribution and history.
- `assignments`: Counterbalanced Latin square client assignment tracking table.

How to Open and Inspect Logs:

1. **Option A: DB Browser for SQLite (Recommended GUI)** - Download and open DB Browser for SQLite. - Open `interaction_portal/logs/portal.sqlite3`. - Navigate to the **Browse Data** tab to view tables, or use **Execute SQL** to run queries.
2. **Option B: Command Line (`sqlite3`)** `bash sqlite3 logs/portal.sqlite3` Useful SQL queries: ```sql -- View recent participant events SELECT timestamp, pid, page_id, event_type, metadata FROM events ORDER BY id DESC LIMIT 20;

-- Check submitted cards with typed share percentage SELECT pid, client_id, timestamp, json_extract(card, '.typed_share') AS typed_share FROM cards;

-- Check post-task check-in responses SELECT timestamp, pid, client_id, json_extract(metadata, '.item_id') AS item, json_extract(metadata, '.value') AS answer FROM events WHERE event_type = 'checkin_answer'; ```

3. **Option C: Direct Web Data Exports & Admin Dashboard** While the server is running, download structured data directly via protected admin endpoints using `PORTAL_ADMIN_TOKEN`. For security, tokens are passed via headers (`x-admin-token` or `Authorization: Bearer <token>`) rather than URL query parameters.

Exporting Flat CSV and JSON Data: ```bash # Export raw events as CSV curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/events.csv -o events.csv

Export flat transcripts CSV (all turns & folded bursts) curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/transcripts.csv -o transcripts.csv

Export panels CSV curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/panels.csv -o panels.csv

Export card spans CSV (one row per span with full history) curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/card_spans.csv -o card_spans.csv

Export wide check-in survey CSV curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/checkins.csv -o checkins.csv

Export sessions summary CSV curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/sessions.csv -o sessions.csv

Export complete database dump as JSON curl.exe -H "x-admin-token: YOUR_ADMIN_TOKEN" http://127.0.0.1:8000/admin/export/all.json -o all_data.json ```

Interactive Admin Dashboard (`/admin`): The `/admin` route provides a live monitoring dashboard for laboratory sessions, displaying: - Active participant sessions (condition, current page, active client, remaining time, error count). - Condition balance counts across all started sessions. - Live model latency and failure metrics. - Abandon/End session controls for stuck participants.

Server Health & Completion Verification: - `GET /admin/health`: Returns JSON with active background queue length and p95 model latency. - `GET /verify?pid=PID&code=CODE` (or `GET /api/verify`): Returns `{valid: true, end_state: "completed"}` for server-side verification by Qualtrics Web Service or researcher scripts.

3.4 `app/harnesses/` -- Automated Verification Test Harnesses

Intention & Purpose: These harnesses provide automated verification of information boundaries and simulation stability **before** running human participants. They are rerun whenever model versions or system prompts are modified.

1. Harness A: Behavioural Rules & Boundary Verification (`app/harnesses/harness_a.py`)

Executes an automated battery of adversarial probes against specialists and the Orchestrator to verify 4 behavioral rules: - **Rule 1 (No Unstated Facts):** Specialists never invent client facts not present in their private materials. - **Rule 2 (No Unasked Disclosure):** Specialists do NOT disclose hidden items during general questions or broadcasts; disclosure only occurs when explicitly asked about that specific topic. - **Rule 3 (No Domain Crossover):** Specialists decline and redirect questions outside their designated domain (e.g. Nia asked about budget declines and redirects to Mira). - **Rule 4 (No Orchestrator Hallucination):** The Orchestrator asserts no client facts, holds no private material, and assigns work based solely on the public brief.

How to run:

```
# Test single client (C1)
python -m app.harnesses.harness_a --config config/config.yaml --client C1

# Test all clients (C1 through C7)
python -m app.harnesses.harness_a --client all
```

Outputs a structured test report (`harness_a_report.json`) with pass/fail status per probe.

2. Harness B: Autopilot End-to-End Simulation (`app/harnesses/harness_b.py`)

Replaces the human participant with an automated scripted driver to execute complete sessions end-to-end: - Exercises orientation, team, role, and scripted practice onboarding (including role checks). - Triggers opening bursts, verifies folded block generation (`blk_*`), and checks reply latencies. - Sends broadcast queries and direct `@mentions` to all specialists. - Validates that panel generation adheres strictly to the 60-80 word budget and condition sections. - Exercises card editor actions per role (passive verification, evaluative keep/cut/send-back, generative typing share). - Submits cards, answers check-ins, and verifies that all emitted telemetry events are tagged with `"autopilot": true`.

How to run:

```
# Test single condition and client
python -m app.harnesses.harness_b --condition EVA-TASK --client C2

# Test all 10 experimental conditions
```

```
python -m app.harnesses.harness_b --all-conditions

# Test complete 7-client session to exit screen
python -m app.harnesses.harness_b --condition GEN-DEV --full-session
```

Outputs a structured execution report (`harness_b_report.json`).

3.5 `tests/` -- Automated Test Suite

Intention & Purpose: The `tests/` directory contains an automated test suite (70 tests across 13 test modules) using `pytest`. It verifies backend mechanics, security guarantees, data integrity, API routes, operational maintenance tools, and boundary invariants.

Test Modules:

- `test_config_loader.py`: Validates YAML parsing, condition mappings, private files, and timer scaling.
- `test_event_store.py`: Tests append-only SQL triggers (rejecting UPDATE/DELETE), atomic single-use session locks, optimistic versioning, and CSV/JSON export formatting.
- `test_state_machine.py`: Tests forward-only page flows, NOAI page skips, break triggers, 7th client alone mode, Latin square condition balancing, and wall-clock deadlines.
- `test_card_provenance.py`: Validates span tracking, keep/cut/send-back actions, and strict generative typed share enforcement.
- `test_chat_router.py`: Tests `@mention` parsing, broadcast aliases, fuzzy typo suggestions, and folded block grouping.
- `test_agents_and_mock.py`: Verifies specialist structured outputs, orchestrator burst planning, and panel word budget guardrails.
- `test_practice.py`: Tests scripted onboarding exercises and wrong role-check redirect penalties.
- `test_session_manager.py`: Tests session resume rules, 15-minute gap timeouts, chat streaming, and check-in validation.
- `test_api_endpoints.py`: Tests FastAPI routes, signed session cookies, desktop vs mobile filtering, rate limiting, and admin token authorization.
- `test_harnesses.py`: Programmatic execution of Harness A and Harness B.
- `test_terminal.py`: Validates ANSI stripping fallback, Windows VT console color detection, and colorize helpers.
- `test_theme.py`: Validates theme styling tokens and UI accessibility rules.
- `test_bugs_and_improvements.py`: Dedicated regression tests for all review findings (lock decoupling, panel validator/trim, strict typed share, opening bursts, randomized check-in option ordering, signed entry links, PII redaction, and database backup verification).

How to run tests:

```
# Run all tests (70 total)
pytest tests -v

# Run a specific test file
pytest tests/test_event_store.py -v

# Run with stdout output enabled
pytest tests -s
```

3.6 `requirements.txt` -- Python Dependencies Breakdown

Intention & Purpose: `requirements.txt` lists all external Python packages required to run and test the portal.

Package	Purpose
`fastapi`	High-performance ASGI web framework providing HTTP routes and NDJSON chat streaming.
`uvicorn`	Production-ready ASGI server for serving the FastAPI application.
`starlette`	Core ASGI toolkit underlying FastAPI (used for request handling and test client).
`jinja2`	Template engine for rendering participant-facing HTML pages.
`python-multipart`	Parser for handling HTML form submissions.

Package	Purpose
`pydantic`	Data validation and schema enforcement for structured LLM outputs.
`pyyaml`	Parser for loading `config.yaml`, `conditions.yaml`, and Markdown frontmatter.
`httpx`	Async HTTP client used by the test client, Claude HTTP streaming, and web requests.
`openai`	Official client library for talking to OpenAI-compatible LLM endpoints (ChatGPT, DeepSeek, GLM, Qwen).
`anthropic`	Official client library for interacting with Anthropic Claude models.
`pytest`	Test execution framework for running the automated unit and integration suite.
`colorama`	Cross-platform colored terminal text support on Windows (Anaconda Prompt, cmd.exe).
`markdown`	Markdown parsing and HTML compilation for documentation export.
`xhtml2pdf`	HTML/CSS to PDF rendering engine for generating standalone PDF documentation.

> **Tip**
> **Anaconda Prompt & Windows Terminal Colors:** The application automatically initializes Windows console color handling at startup (`app.terminal.init_terminal()`). In Anaconda Prompt and `cmd.exe`, it activates Windows Virtual Terminal (VT) processing and `colorama` so that Uvicorn server logs and harness reports display in vibrant colors without raw escape artifacts (such as `<-[32m`, `<-[0m`, `<-[36m`, `<-[1m`). If colors are not supported (e.g. redirected log files or non-VT terminals), a built-in fallback cleanly omits and strips all ANSI escape codes.

Installation:

```
pip install -r requirements.txt
```

3.7 `md_to_pdf.py` -- Markdown-to-PDF Documentation Exporter

Intention & Purpose: `md_to_pdf.py` converts Markdown documentation files into styled, publication-ready A4 PDF documents. It formats headings, tables, callout banners, and code blocks with running headers, footers, and page numbers.

Usage:

```
# Convert README.md to README.pdf (default)
python md_to_pdf.py

# Convert any custom Markdown file
python md_to_pdf.py docs/guide.md docs/guide.pdf
```

---

4. Production Deployment & Database Administration

This section covers laboratory production configuration with Qualtrics, server security hardening, reverse proxy setup, and automated database maintenance. If you simply want to test or demo the application locally, refer to the Quick Start Guide at the top of this document.

4.1 Production Deployment with Qualtrics

In production, participants arrive via Qualtrics with their assigned `pid` and condition code in the query parameters:

```
http://<host>:8000/?pid=R_123456789&condition=EVA-TASK
```

Production Security Requirements:

When `PORTAL_STANDALONE` is `0` (or `PORTAL_ENV=production`), the application enforces strict production safeguards at startup. The server will refuse to start if secrets equal known development defaults or are shorter than 24 characters. Ensure you configure cryptographically random strings for: - `PORTAL_SECRET_KEY` (minimum 24 characters) - `PORTAL_ADMIN_TOKEN` (minimum 24 characters) - `PORTAL_COMPLETION_SECRET` (minimum 24 characters)

Setting Environment Variables:

On Linux / macOS:

```
# Disable standalone mode (requires pid and condition query parameters)
export PORTAL_STANDALONE="0"

# Configure LLM provider (openai, claude, deepseek, glm, qwen) and corresponding API key
export PORTAL_LLM_PROVIDER="openai"
export OPENAI_API_KEY="sk-..."
# (Or set ANTHROPIC_API_KEY, DEEPSEEK_API_KEY, GLM_API_KEY, or DASHSCOPE_API_KEY)

# Configure secure session cookie signing key and data export token
export PORTAL_SECRET_KEY="generate-a-secure-random-secret-key-min-24-chars"
export PORTAL_ADMIN_TOKEN="generate-a-secure-admin-token-min-24-chars"
export PORTAL_COMPLETION_SECRET="generate-a-secure-completion-secret-min-24-chars"

# Optional: Host and port (default is 127.0.0.1:8000)
# export PORTAL_HOST="127.0.0.1"
# export PORTAL_PORT="8000"

# Launch using production single-worker ASGI server
uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 1
```

In Windows PowerShell:

```
# Disable standalone mode (requires pid and condition query parameters)
env:PORTAL_STANDALONE="0"

# Configure LLM provider (openai, claude, deepseek, glm, qwen) and corresponding API key
env:PORTAL_LLM_PROVIDER="openai"
env:OPENAI_API_KEY="sk-..."
# (Or set env:ANTHROPIC_API_KEY, env:DEEPSEEK_API_KEY, env:GLM_API_KEY, or env:DASHSCOPE_API_KEY)

# Configure secure session cookie signing key and data export token
env:PORTAL_SECRET_KEY="generate-a-secure-random-secret-key-min-24-chars"
env:PORTAL_ADMIN_TOKEN="generate-a-secure-admin-token-min-24-chars"
env:PORTAL_COMPLETION_SECRET="generate-a-secure-completion-secret-min-24-chars"

# Optional: Host and port (default is 127.0.0.1:8000)
# env:PORTAL_HOST="127.0.0.1"
# env:PORTAL_PORT="8000"

python -m app.main
```

In Windows Command Prompt (cmd.exe):

```
:: Disable standalone mode (requires pid and condition query parameters)
set PORTAL_STANDALONE=0

:: Configure LLM provider (openai, claude, deepseek, glm, qwen) and corresponding API key
set PORTAL_LLM_PROVIDER=openai
set OPENAI_API_KEY=sk-...
:: (Or set ANTHROPIC_API_KEY, DEEPSEEK_API_KEY, GLM_API_KEY, or DASHSCOPE_API_KEY)

:: Configure secure session cookie signing key and data export token
set PORTAL_SECRET_KEY=generate-a-secure-random-secret-key-min-24-chars
set PORTAL_ADMIN_TOKEN=generate-a-secure-admin-token-min-24-chars
set PORTAL_COMPLETION_SECRET=generate-a-secure-completion-secret-min-24-chars

python -m app.main
```

Production Architecture & Deployment Considerations:

1. **Single-Worker Requirement (`WEB_CONCURRENCY=1`, `uvicorn --workers 1`):** Participant session state, optimistic versioning, lock decoupling, and background exchange orchestration are managed in-process. The server strictly enforces single-worker execution at startup and rejects startup if `WEB_CONCURRENCY > 1`. Scaling across concurrent participants in a laboratory run is

handled by FastAPI's asynchronous event loop and decoupled thread pool, not multiple OS worker processes.

2. **Reverse Proxy Configuration (Nginx / Caddy):** - **NDJSON Chat Streaming:** Streaming endpoints send `X-Accel-Buffering: no` headers. In Nginx, configure `proxy_buffering off;` and `proxy_read_timeout 120s;` for the `/api/chat/stream` path to prevent buffering of live streaming tokens. - **Forwarded Headers:** When running behind a reverse proxy, invoke Uvicorn with `--proxy-headers --forwarded-allow-ips "*"` so client IP addresses and protocol headers are propagated. - **HTTPS & Secure Cookies:** If the reverse proxy terminates TLS and forwards plain HTTP upstream to Uvicorn, set `PORTAL_COOKIE_SECURE=1` (or `PORTAL_COOKIE_SECURE=true`) to ensure session cookies are marked `Secure`.

3. **Signed Entry Links & Session Recovery:** - **HMAC Signatures:** To prevent condition tampering, production URLs can be signed using `qualtrics.completion_code_secret`: `text http://<host>:8000/?pid=R_123456789&condition=EVA-TASK&exp=1770000000&sig=<hmac_sha256>` where `sig` is computed as `HMAC-SHA256(secret, f"{pid}|{condition}|{exp}")`. The server rejects expired or tampered signatures with an informative error. - **Session Recovery:** Each session generates a cryptographically random `resume_token`. If a participant accidentally closes their browser tab or clears session cookies, they can resume their session within `timers.resume_gap_seconds` (default 15 minutes) via: `text http://<host>:8000/?resume=<resume_token>`

4. **Server-Side Completion Verification:** At the end of the session, the portal redirects participants back to Qualtrics: `text https://example.qualtrics.com/jfe/form/SV_demo?pid=R_123456789&code=IP2-A1B2C3D4` External systems or Qualtrics Web Service survey blocks can verify completion authenticity server-side: `bash curl -H "x-admin-token: YOUR_ADMIN_TOKEN" "http://127.0.0.1:8000/verify?pid=R_123456789&code=IP2-A1B2C3D4"` Returns: `json {"valid": true, "end_state": "completed", "verify_count": 1}`

4.2 Database Maintenance, Backups & Verification

The portal includes dedicated administrative CLI tools in `app/tools/` for automated operational maintenance.

1. Automated Nightly Backups (`app.tools.backup`)

Performs an online, non-blocking backup of the telemetry SQLite database using SQLite's native backup API, verifies database integrity (`PRAGMA integrity_check`), compresses the snapshot with gzip, generates a SHA-256 verification manifest, and rotates older backups based on retention policy:

```
python -m app.tools.backup --dest backups/ --keep 14
```

Automating with Cron (Linux / macOS): Add to `crontab -e`:

```
0 2 * * * cd /path/to/interaction_portal && /path/to/venv/bin/python -m app.tools.backup --dest backups/ --keep 14
```

Automating with Windows Task Scheduler: 1. Open Task Scheduler \to **Create Basic Task**. 2. Trigger: Daily at 02:00. 3. Action: Start a program. - Program: `D:\Desktop\Git\multi_agent_chat\.venv\Scripts\python.exe` - Arguments: `-m app.tools.backup --dest backups/ --keep 14` - Start in: `D:\Desktop\Git\multi_agent_chat\interaction_portal`

2. Database Integrity Verification (`app.tools.verify_db`)

Inspects the active database to ensure data integrity and structural invariants:

```
python -m app.tools.verify_db
```

Checks performed: - `PRAGMA integrity_check` and `PRAGMA foreign_key_check`. - Existence of append-only database triggers (`events_no_update`, `events_no_delete`). - Schema consistency across all required tables (`events`, `sessions`, `transcripts`, `panels`, `cards`, `assignments`). - Strict monotonic timestamp ordering per participant session.

3. Participant PII Purging Procedure (`app.tools.purge_pii`)

If a participant inadvertently enters personally identifiable information (such as an email, phone number, IBAN, credit card, or name) that must be redacted under data protection compliance, run the maintenance purge script:

```
python -m app.tools.purge_pii
```

This utility: 1. Scans transcript texts and event metadata for PII patterns. 2. Begins an atomic database transaction. 3. Temporarily suspends append-only triggers, redacts matching substrings with `[REDACTED_PII]`, logs an administrative audit event, and re-establishes append-only triggers. 4. Commits the transaction and verifies trigger restoration.